

HERMES AGENT | AULA 03

Como montar e testar o Hermes avançado

Profiles, memória externa, OpenViking, Hindsight, Codex, subagentes, Kanban, Cron e A2A

Bloco 1 - Profiles e sessões separadas

Profiles são instâncias independentes do Hermes dentro da mesma instalação. Cada Profile possui configuração, sessões, memória, skills, cron jobs e estado próprios.

Acesso remoto e múltiplos usuários com autenticação serão tratados na Aula 4, junto com criação e uso de Skills e integração com MCP.

Recomendação para esta aula: use a linha de comando para iniciar cada Profile. Assim, o Hermes recebe explicitamente o Profile no momento em que o processo é criado e reduz o risco de uma sessão herdar estado, diretório de trabalho ou contexto de outro Profile.

O Desktop e o Dashboard continuam úteis para administração e visualização, mas não serão o caminho principal para abrir as sessões dos Profiles nesta aula. Para iniciar uma sessão, prefira: `hermes -p <profile> chat`

Antes de começar - se o Hermes Desktop aparecer, mas não carregar no WSL2

No WSL2, o Hermes Desktop depende da integração gráfica do Linux com o Windows por meio do WSLg. Em alguns computadores, a janela pode aparecer na barra de tarefas ou surgir com um aviso, mas a interface não carregar corretamente.

Primeiro, encerre possíveis processos travados no terminal Ubuntu:

```
pkill -f Hermes
pkill -f electron
pkill -f chromium
```

Depois, feche o Ubuntu. No PowerShell do Windows, execute:

```
wsl --shutdown
```

Abra novamente o Ubuntu e tente iniciar o Desktop:

```
cd ~/hermes-lab
hermes desktop
```

Este procedimento reinicia o ambiente gráfico/processos do WSL2 e é diferente do problema de cwd/Profile descrito mais adiante nesta aula. Se o Desktop continuar sem carregar, a demonstração pode seguir pela CLI ou pelo Dashboard enquanto o problema gráfico é diagnosticado.

1.1 Criar o Profile aluno

1. Abrir o Dashboard do Hermes.

2. Acessar Profiles.
3. Selecionar Create Profile.
4. Criar o Profile com o nome aluno.
5. Adicionar uma descrição curta para identificar o Profile.

1.2 Configurar o Profile aluno

Se o Profile ainda não tiver modelo/provedor configurado, executar o setup:

```
hermes -p aluno setup
```

Depois do setup, iniciar o Profile aluno diretamente pelo terminal:

```
hermes -p aluno chat
```

1.3 Iniciar Profiles pela linha de comando

Sempre que for abrir ou trocar de Profile durante a demonstração, encerre a sessão atual e inicie explicitamente o Profile desejado pelo terminal.

```
hermes -p aluno chat
```

```
hermes -p default chat
```

1.4 Confirmar sessões separadas

6. Iniciar uma nova sessão do Profile aluno pelo terminal com `hermes -p aluno chat`.
7. Enviar uma mensagem simples para identificar essa sessão.
8. Encerrar a sessão atual e iniciar o Profile principal pelo terminal com `hermes -p default chat`.
9. Confirmar que a sessão criada no Profile aluno não aparece no Profile principal.
10. Voltar ao Profile aluno com `hermes -p aluno chat` e confirmar que a sessão anterior continua disponível.

1.5 Atenção ao cwd no Hermes Desktop

O Hermes Desktop pode manter um diretório de trabalho (cwd) antigo entre sessões ou trocas de Profile. Nessa situação, alterar `terminal.cwd` no `config.yaml` pode não ser suficiente, porque o Desktop também mantém o valor lembrado em `hermes.desktop.workspace-cwd`.

Consequência: a nova sessão pode abrir no Workspace anterior e carregar arquivos de contexto, skills, memória ou configurações associadas ao projeto errado. Por isso, para a demonstração de Profiles, a linha de comando é o caminho preferido.

Correção no Desktop, quando o problema aparecer: fechar o Hermes Desktop, confirmar o `terminal.cwd` correto no `config.yaml` do Profile e remover a chave `hermes.desktop.workspace-cwd` do armazenamento local do Electron. No Windows, esse armazenamento fica em `%APPDATA%\Hermes\Local Storage\leveldb\`. Depois, abrir novamente o Desktop e criar uma nova sessão.

Esse procedimento é específico do Desktop. Problemas de Profile/cwd no Dashboard ou em gateways long-lived têm outra origem e não devem ser corrigidos apagando o LevelDB do Desktop.

Bloco 2 - Memória persistente do Hermes

A memória embutida do Hermes mantém informações úteis entre sessões e é separada por Profile. Ela usa MEMORY.md e USER.md internamente, mas a demonstração pode ser feita diretamente pelo chat.

2.1 Demonstrar a memória entre sessões

11. No Profile aluno, iniciar uma nova conversa.
12. Informar uma preferência simples, por exemplo: "Prefiro exemplos em Python quando houver código."
13. Encerrar a sessão e abrir outra sessão no mesmo Profile.
14. Perguntar qual linguagem deve ser priorizada nos exemplos.

Bloco 3 - Memória externa: conceito antes da instalação

O Hermes possui memória interna e também permite conectar provedores de memória externa. A memória externa mantém conhecimento persistente além da memória embutida e pode recuperar somente a informação relevante para cada pergunta.

A memória interna continua ativa junto com o provedor externo. Porém, cada Profile pode ter apenas um provedor externo ativo por vez.

3.1 Por que memória externa pode reduzir o uso de contexto

Quando documentos grandes são colocados diretamente na conversa, parte ou todo o conteúdo passa a ocupar a janela de contexto do modelo. Com recuperação sob demanda, o sistema procura primeiro a informação relevante e envia ao modelo somente o trecho necessário para responder.

Isso não aumenta a janela máxima do modelo. O ganho está em evitar que grandes volumes de informação permaneçam no contexto a cada interação, o que pode reduzir o consumo de tokens de contexto.

3.2 OpenViking e Hindsight resolvem problemas diferentes

- OpenViking: organiza conhecimento em uma hierarquia de recursos e usa carregamento em camadas. É especialmente útil para recuperar partes específicas de documentos sem carregar tudo de uma vez.
- Hindsight: trabalha com memória de longo prazo, entidades, relações e síntese/reflexão sobre fatos armazenados. A vantagem principal é estruturar e relacionar conhecimento; a economia de contexto pode ocorrer por recuperação sob demanda, mas não é sua função principal.

Como somente um provedor externo pode ficar ativo por Profile, o Profile aluno será usado com OpenViking e o Profile lois será usado com Hindsight.

Bloco 4 - OpenViking como memória externa

4.1 Instalar o uv no Ubuntu/WSL2

Instale primeiro o uv pelo instalador oficial da Astral. Execute no terminal do Ubuntu/WSL2:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
source ~/.bashrc
uv --version
```

Se `uv --version` exibir a versão instalada, prossiga. Se o comando ainda não for encontrado, feche e abra o terminal do Ubuntu e teste novamente.

Evite usar `sudo snap install astral-uv --classic` como caminho principal. O instalador standalone da Astral evita o aviso de classic confinement do Snap e segue a documentação oficial do uv.

4.2 Instalar e preparar o OpenViking

Escolha a forma de instalação de acordo com o tipo de embedding que será usado no OpenViking.

Opção 1 - Instalação padrão

Use esta opção quando o embedding será configurado por Cloud API ou por Ollama.

```
uv tool install openviking --upgrade
```

Opção 2 - Embedding leve na CPU

Use esta opção quando quiser selecionar Lightweight CPU embedding no assistente, sem depender de Ollama ou de uma API externa para o embedding.

```
uv tool install "openviking[local-embed]" --upgrade
```

O extra `local-embed` adiciona a dependência necessária para o embedding local baseado em `llama.cpp`.

Se o OpenViking já tiver sido instalado pela Opção 1 e depois você decidir usar Lightweight CPU embedding, execute o comando da Opção 2 para atualizar a instalação com esse extra.

Depois da instalação, inicie o assistente de configuração:

```
openviking-server init
```

No assistente, escolha Step-by-step setup. Em Embedding setup, selecione Cloud API, Local via Ollama ou Lightweight CPU embedding de acordo com a opção de instalação escolhida acima.

No passo VLM, escolha o provedor que será usado para interpretação e síntese. Um fluxo testado é OpenAI Codex (Subscription), que usa autenticação da assinatura; outras opções podem ser usadas conforme o ambiente.

Se Lightweight CPU embedding informar que `llama-cpp-python` não está instalado, não continue com uma configuração incompleta. Saia do assistente, execute `uv tool install "openviking[local-embed]" --upgrade` e rode `openviking-server init` novamente.

Após concluir o assistente, valide a configuração:

```
openviking-server doctor
```

4.3 Usar o servidor local sem API key

Quando o OpenViking roda somente em localhost, com host 127.0.0.1, a API key não é obrigatória. Sem `root_api_key` configurada, o servidor entra automaticamente em modo de desenvolvimento (dev mode), com autenticação desativada.

Configuração local mínima em `~/openviking/ov.conf`:

```
{
  "server": {
    "host": "127.0.0.1",
    "port": 1933
  }
}
```

Não configure `root_api_key` como string vazia. Para uso local, remova o campo. Se o servidor for exposto para a rede ou para outro computador, configure autenticação com uma `root_api_key` antes de alterar o host para um endereço não local.

4.4 Iniciar o servidor OpenViking

`openviking-server`

O servidor usa por padrão a configuração em `~/openviking/ov.conf`. Na primeira configuração, `init` prepara o arquivo e `doctor` verifica se o ambiente local está utilizável.

Mantenha esse terminal aberto enquanto o Hermes estiver usando o OpenViking. O servidor normalmente inicia na porta 1933.

4.5 Ativar OpenViking local no Profile aluno

Abra um segundo terminal Ubuntu. Mantenha o terminal do `openviking-server` aberto.

```
hermes -p aluno config set memory.provider openviking
```

Configure o endpoint local do OpenViking no arquivo `.env` do Profile aluno:

```
echo "OPENVIKING_ENDPOINT=http://127.0.0.1:1933" >> ~/.hermes/profiles/aluno/.env
```

O assistente `hermes -p aluno memory setup` pode exigir uma API key mesmo quando o servidor local está em dev mode. Para uso local sem autenticação, configure o provider e o endpoint manualmente com os comandos acima.

```
hermes -p aluno memory status
```

Use `memory status` para confirmar que o provider OpenViking ficou ativo no Profile aluno.

Importante: cada Profile pode usar sua própria configuração de memória externa. Nesta aula, o Profile aluno será usado com OpenViking e o Profile lois será usado com Hindsight.

A memória persistente nativa do Hermes continua existindo; o provedor externo adiciona uma camada de armazenamento e recuperação especializada.

4.6 Adicionar e consultar conhecimento

15. Iniciar uma nova sessão do Profile aluno pelo terminal com `hermes -p aluno chat`.
16. Adicionar um recurso pequeno ao OpenViking usando as ferramentas disponibilizadas pelo provedor.
17. Fazer uma pergunta específica sobre o conteúdo adicionado.
18. Solicitar a indicação do recurso ou URI consultado.

Bloco 5 - Hindsight e memória de longo prazo

Hindsight será usado no Profile lois porque o Profile aluno já possui OpenViking como provedor externo ativo.

5.1 Preparar o Profile lois

Se o Profile lois já existir, não é necessário criá-lo novamente. Se ainda não existir, crie o Profile antes de continuar.

Se o Profile lois ainda não tiver modelo/provedor configurado, execute:

```
hermes -p lois setup
```

5.2 Instalar o Hindsight Local Embedded manualmente

Para evitar o timeout do instalador automático, instale previamente o pacote completo do modo Local Embedded no ambiente Python do Hermes:

```
uv pip install --python ~/.hermes/hermes-agent/venv/bin/python hindsight-all
```

O assistente do Hermes pode tentar instalar hindsight-all automaticamente. Em alguns ambientes, essa instalação ultrapassa o limite de 120 segundos e exibe `uv pip install timed out`. A instalação manual acima evita depender desse timeout.

5.3 Configurar o Hindsight no Profile lois

Execute:

```
hermes -p lois memory setup
```

No menu, selecione Hindsight e depois Local Embedded - Run Hindsight locally.

Quando solicitado, escolha o provedor de LLM. O fluxo testado utiliza MiniMax. No modo Local Embedded, embeddings e reranking são executados localmente.

Se o assistente ainda tentar instalar hindsight-all e exibir timeout, conclua ou cancele a etapa, execute novamente o comando manual de instalação e repita `hermes -p lois memory setup`.

5.4 GPUs antigas ou incompatibilidade CUDA: forçar CPU

Se retain ou recall falharem com `cudaErrorNoKernelImageForAvailable`, o backend local de embeddings/reranking tentou usar uma GPU CUDA incompatível com os kernels disponíveis.

Primeiro configure o runtime embedded do Hindsight para CPU:

```
hindsight-embed profile create hermes --merge --env  
HINDSIGHT_API_EMBEDDINGS_LOCAL_FORCE_CPU=true --env  
HINDSIGHT_API_RERANKER_LOCAL_FORCE_CPU=true
```

Se for necessário registrar as mesmas variáveis no Profile lois, execute:

```
echo "HINDSIGHT_API_EMBEDDINGS_LOCAL_FORCE_CPU=true" >> ~/.hermes/profiles/lois/.env  
echo "HINDSIGHT_API_RERANKER_LOCAL_FORCE_CPU=true" >> ~/.hermes/profiles/lois/.env
```

Confira:

```
tail -n 10 ~/.hermes/profiles/lois/.env
```

Depois feche sessões do Hermes/Hindsight em execução e abra novamente:

```
hermes -p lois chat
```

5.5 Confirmar o Hindsight ativo

```
hermes -p lois memory status
```

O status deve indicar Hindsight como provedor externo ativo no Profile lois.

5.6 Testar retain, recall e reflect

Abra o Profile lois:

```
hermes -p lois chat
```

No chat, peça explicitamente para usar `hindsight_retain` e armazenar uma informação de teste. Em seguida, encerre a sessão, abra uma nova sessão e peça para usar `hindsight_recall` para recuperar a informação.

Para testar síntese entre memórias, armazene duas informações relacionadas e peça para usar `hindsight_reflect` para resumir as relações encontradas.

Observação: `retain` pode ser assíncrono; aguarde alguns segundos antes do primeiro `recall` quando a memória tiver acabado de ser gravada.

5.7 Criar relações e recuperar conhecimento

Registrar fatos relacionados, por exemplo: Ana coordena o Curso Hermes; Bruno mantém uma API usada pelo curso; o curso usa OpenViking para documentação.

Pedir ao agente para recuperar as relações entre as entidades registradas.

Pedir uma reflexão separando fatos registrados de inferências.

Bloco 6 - Usar Codex no mesmo Profile aluno

Não é necessário criar um Profile exclusivo para Codex. O runtime Codex pode ser ativado no mesmo Profile aluno usado com OpenViking. O Profile continua sendo o mesmo; o que muda é o runtime utilizado para aquele tipo de sessão.

No runtime `codex_app_server`, ferramentas que dependem diretamente do loop interno do Hermes, como `delegate_task` e a ferramenta `memory`, não ficam disponíveis durante o turno. Depois da

demonstração de Codex, voltar para o runtime padrão antes de continuar com memória ou subagentes.

6.1 Instalar e autenticar o Codex CLI

Executar no terminal:

```
npm i -g @openai/codex
codex --version
codex login
```

Usar Codex CLI 0.130.0 ou superior.

6.2 Preparar um Workspace para o teste

```
mkdir -p ~/hermes-lab/aula3-codex
cd ~/hermes-lab/aula3-codex
pwd
```

19. Permanecer no diretório retornado por pwd.
20. Iniciar o Profile aluno a partir desse Workspace pelo terminal.
21. Executar: hermes -p aluno chat

6.3 Ativar o runtime Codex

No chat do Profile aluno:

```
/codex-runtime
/codex-runtime codex_app_server
```

Abrir uma nova sessão após a ativação.

6.4 Executar uma tarefa de código

22. Pedir ao Hermes para confirmar o diretório atual e listar os arquivos do Workspace.
23. Pedir a criação de um pequeno projeto Python com arquivo principal, testes e README.
24. Acompanhar as alterações diretamente no diretório do Workspace pelo terminal ou editor de arquivos.
25. Executar os testes pelo terminal apenas se necessário.

6.5 Voltar ao runtime padrão

```
/codex-runtime auto
```

Abrir uma nova sessão antes dos próximos blocos.

Bloco 7 - Subagentes

Subagentes são execuções temporárias criadas por `delegate_task`. Eles recebem uma tarefa e um contexto isolado, executam o trabalho e devolvem o resultado ao agente principal. Não são Profiles persistentes.

7.1 Demonstrar um subagente

26. No terminal, usar uma sessão do Profile desejado iniciada com `hermes -p <profile> chat` e manter o runtime padrão.

27. Pedir ao agente: "Delegue a um subagente a tarefa de comparar memória persistente e contexto de conversa. Retorne somente o resumo final."
28. Acompanhar a execução pela interface de agentes quando disponível.

Bloco 8 - Kanban: criar e acompanhar um projeto

O Kanban organiza tarefas duráveis em um quadro e registra o andamento do trabalho. Nesta demonstração, todas as etapas serão executadas pelo Profile aluno já criado anteriormente.

8.1 Abrir o Kanban

29. Abrir Dashboard > Kanban.
30. Se o banco ainda não existir, o próprio acesso ao Kanban pode inicializá-lo automaticamente. Se for necessário inicializar manualmente:
`hermes kanban init`

8.2 Criar o projeto no Kanban

Projeto: pesquisar uma atualização técnica do Hermes, produzir um resumo didático e revisar o resultado antes da conclusão.

31. Manter Orchestration em Manual no primeiro teste.
32. Criar uma tarefa principal em Triage para representar o projeto.
33. Criar as etapas do trabalho: pesquisar o tema, produzir o material e revisar o resultado.
34. Definir as dependências entre as etapas.
35. Atribuir todas as tarefas ao Profile aluno.
36. Conferir objetivo, Workspace e critérios de aceitação de cada tarefa.
37. Mover a primeira tarefa para Ready.
38. Usar Nudge dispatcher para executar um ciclo imediatamente, quando necessário.
39. Acompanhar a tarefa em Ready, Running e Done ou Blocked.
40. Quando uma etapa for concluída, acompanhar a liberação da próxima tarefa dependente.
41. Repetir o fluxo até a conclusão do projeto.
42. Abrir os cartões para visualizar histórico, execução, comentários e resultado.

8.3 Testar a orquestração automática

43. Depois de confirmar o fluxo manual, criar outra tarefa principal em Triage.
44. Usar Decompose para gerar subtarefas.
45. Conferir se as subtarefas e dependências fazem sentido e se o Profile aluno foi mantido como responsável.
46. Somente depois dessa conferência, testar Orchestration: Auto.

Bloco 9 - Cron e automações

Cron permite criar tarefas únicas ou recorrentes. A criação pode ser feita em linguagem natural pelo próprio agente e acompanhada pelo Desktop ou Dashboard.

9.1 Criar uma automação

47. Abrir uma conversa pelo terminal com o Profile escolhido, por exemplo: `hermes -p aluno chat`.
 48. Pedir: "Daqui a 5 minutos, crie um resumo curto do estado atual deste projeto e entregue localmente."
 49. Abrir a área Cron no Hermes Desktop ou Dashboard.
 50. Confirmar nome, agenda, estado e destino.
 51. Usar Trigger now para testar imediatamente.
 52. Testar Pause, Resume, Edit e Delete pela interface.
- Cron jobs executam em sessões novas. O prompt deve conter o contexto necessário para executar a tarefa sozinho.

Bloco 10 - A2A: comunicação entre agentes independentes

A2A é o protocolo de comunicação entre agentes independentes. Um Hermes pode chamar outro Hermes como agente, descobrir suas capacidades e manter uma conversa A2A por `context_id`.

A2A não é obrigatório para o Kanban. Para vários Profiles na mesma máquina, o Kanban já fornece uma fila durável de trabalho. A2A é mais indicado quando a comunicação precisa atravessar processos, máquinas ou frameworks independentes.

10.1 Exemplo: Profile principal chamando o Profile lois

O exemplo usa o Profile principal como cliente A2A e o Profile lois como agente chamado. O mesmo fluxo pode ser reproduzido com outros nomes de Profiles.

10.2 Habilitar A2A no Profile lois

Executar o setup do gateway do Profile lois e habilitar A2A:

```
hermes -p lois gateway setup
```

Definir uma porta local exclusiva para o A2A do Profile lois, por exemplo 9901.

Iniciar o gateway do Profile lois explicitamente pelo terminal:

```
hermes -p lois gateway run
```

10.3 Habilitar as ferramentas A2A no Profile principal

53. No Profile principal, abrir a configuração de Tools/Toolsets.
54. Habilitar o toolset a2a.
55. Cadastrar o Profile lois como agente conhecido apontando para o endpoint A2A local.

10.4 Chamar o agente lois

56. No chat do Profile principal iniciado pela linha de comando, pedir para descobrir as capacidades do agente lois.
57. Enviar uma tarefa curta para lois.
58. Fazer uma segunda pergunta mantendo o mesmo `context_id` para demonstrar continuidade da conversa entre agentes.
59. Consultar o histórico da conversa A2A quando necessário.

Bloco 11 - Limpar provedores e excluir Profiles

11.1 Desativar o Hindsight no Profile lois

Antes de remover arquivos locais, volte o Profile lois para a memória embutida:

```
hermes -p lois memory off
```

A saída esperada é Memory provider: built-in only.

11.2 Remover componentes locais do Hindsight

Remova o cliente instalado pelo setup, quando presente:

```
uv pip uninstall --python ~/.hermes/hermes-agent/venv/bin/python hindsight-client
```

Se hindsight-all estiver instalado no mesmo ambiente, remova também:

```
uv pip uninstall --python ~/.hermes/hermes-agent/venv/bin/python hindsight-all
```

Se o comando informar que hindsight-all não está instalado, prossiga normalmente.

Apague somente a instância PostgreSQL embedded criada pelo Hindsight:

```
rm -rf ~/.pg0/instances/hindsight-embed-hermes
```

Remova as variáveis HINDSIGHT_* do Profile lois:

```
sed -i '/^HINDSIGHT_/d' ~/.hermes/profiles/lois/.env
```

Verifique se não restaram variáveis:

```
grep HINDSIGHT ~/.hermes/profiles/lois/.env
```

11.3 Remover o OpenViking instalado pelo uv

```
uv tool uninstall openviking
```

Confirme que não aparece mais na lista:

```
uv tool list
```

Para apagar também configuração e dados locais do OpenViking:

```
rm -rf ~/.openviking
```

Para apagar o cache/modelo local baixado pelo OpenViking, quando quiser zerar completamente o ambiente:

```
rm -rf ~/.cache/openviking
```

11.4 Excluir Profiles

O comando abaixo remove permanentemente o Profile indicado, incluindo configuração, memória, sessões e skills daquele Profile. O Profile default não deve ser removido.

```
hermes profile delete <nome-do-profile> --yes
```

Exemplos usados nesta aula:

```
hermes profile delete aluno --yes
```

```
hermes profile delete lois --yes
```

Confirme pela CLI:

```
hermes profile list
```

Se aluno ou lois não aparecerem mais na lista, a exclusão foi concluída. Se o Dashboard ainda mostrar uma entrada stopped, recarregue ou reinicie o Dashboard para atualizar a interface.

Aula 4

Acesso remoto e múltiplos usuários, criação e uso de Skills e integração com servidores MCP.

Referências oficiais

- Profiles: <https://hermes-agent.nousresearch.com/docs/user-guide/profiles/>
- Profile Commands: <https://hermes-agent.nousresearch.com/docs/reference/profile-commands/>
- Issue Hermes Desktop cwd #38855: <https://github.com/NousResearch/hermes-agent/issues/38855>
- Issue Hermes Desktop profile/cwd #79406:
<https://github.com/NousResearch/hermes-agent/issues/79406>
- Issue Hermes Profiles locais #81492:
<https://github.com/NousResearch/hermes-agent/issues/81492>
- Memória: <https://hermes-agent.nousresearch.com/docs/user-guide/features/memory/>
- Memory Providers: <https://hermes-agent.nousresearch.com/docs/user-guide/features/memory-providers/>

OpenViking Quick Start - Server Mode:

<https://github.com/volcengine/OpenViking/blob/main/docs/en/getting-started/03-quickstart-server.md>

- Codex App-Server Runtime:
<https://hermes-agent.nousresearch.com/docs/user-guide/features/codex-app-server-runtime>
- Subagent Delegation:
<https://hermes-agent.nousresearch.com/docs/user-guide/features/delegation/>
- Kanban: <https://hermes-agent.nousresearch.com/docs/user-guide/features/kanban>
- Kanban Tutorial: <https://hermes-agent.nousresearch.com/docs/user-guide/features/kanban-tutorial>
- Cron: <https://hermes-agent.nousresearch.com/docs/user-guide/features/cron/>
- A2A: <https://hermes-agent.nousresearch.com/docs/user-guide/messaging/a2a>

Hindsight plugin: <https://github.com/NousResearch/hermes-agent/blob/main/plugins/memory/hindsight/README.md>

Hindsight FORCE_CPU example: <https://github.com/NousResearch/hermes-agent/issues/8972>

Hindsight environment variables: <https://github.com/vectorize-io/hindsight/blob/main/.env.example>

OpenViking Quick Start: <https://github.com/volcengine/OpenViking/blob/main/docs/en/getting-started/03-quickstart-server.md>